



COMSATS University Islamabad
Department of Computer Science

Lab Assignment 1 [Marks 10]

Class: BSAI-6.

Subject: ANN & DL

Submission Deadline: **Lab in this week**

Note: Bring along codes of all given tasks in MS-word document along with input. Mention your name, registration number and class on the title page.

Tasks-1

1. Apply Macclouch Pitts neural model to solve logical gates given below.
2. Implement a generic M-P neuron function `mp_neuron(inputs, weights, threshold)`.
3. Using the M-P neuron, realize the following logic gates by manually choosing weights and threshold — show your derivation for each:
 - AND gate (2-input)
 - OR gate (2-input)
 - NOT gate (1-input, use a negative weight)
 - AND gate for 3 inputs
4. Attempt to realize the **XOR gate** using a single M-P neuron.
5. Print a truth table for each gate showing inputs, weighted sum, and output.

Tasks-2

1. Implement a `Perceptron` class with `fit(X, y, epochs, lr)` and `predict(X)` methods, using a step activation function.
2. Train it on the **AND** and **OR** truth tables. Plot the decision boundary after training (2D plot with the separating line).
3. Plot the number of misclassifications per epoch to show convergence.
4. Attempt to train the same perceptron on the **XOR** truth table.
 - Show that it fails to converge (plot the error curve — it will not go to zero).
 - Explain the failure in terms of linear separability (include a sketch/plot of the XOR points showing no straight line can separate them).

Tasks-3

Implement a 2-layer MLP from scratch (input → hidden layer with sigmoid → output layer with sigmoid) using NumPy — no deep learning frameworks.

1. Implement forward propagation and backpropagation (derive and code the gradients manually).
2. Train the MLP on the **XOR** problem until the loss converges near zero. Use at least 2 hidden neurons.

3. Plot:
 - Loss (mean squared error) vs. epoch
 - The learned decision boundary (should now correctly separate XOR)
4. Experiment: vary the number of hidden neurons (2, 4, 8) and the learning rate (0.01, 0.1, 1.0). Tabulate how many epochs each configuration needs to converge.

Tasks-4

1. Modify your MLP from Part C so the initialization scheme is a parameter (`init_method = "zero" | "random" | "xavier" | "he"`).
2. Train the same XOR MLP under each initialization scheme, keeping all other hyperparameters fixed.
3. For **zero initialization**, show and explain that the two hidden neurons learn identical weights (symmetry breaking failure) — print the hidden weight matrix after a few epochs to demonstrate this.
4. Plot the loss curves for all four initialization schemes on the same graph for comparison.
5. Tabulate epochs-to-convergence (or final loss after a fixed number of epochs) for each scheme.